

Technical Report: RoboSubSim Autonomous Agent

1.0 Project Overview

Project Goal: The objective of this project was to develop a robust autonomous software agent capable of navigating a simulated underwater environment and completing a sequence of complex tasks as defined by robotics competition rules. The project involved creating the agent's logic, a physics-based simulator for testing, and iteratively debugging the agent's behavior to achieve a successful mission run.

The Software: The final product is a Python-based simulation environment. It utilizes the Pygame library for visualization and the high-performance OpenCV library for all computer vision and image processing tasks. The core of the project is a modular, state-machine-driven autonomous agent designed to be extensible and robust. The agent's logic is decoupled from the simulation environment, allowing for independent development and testing.

The Mission Tasks: The agent is required to complete two primary objectives in sequence:

1. **Gate Passage:** The submarine must autonomously find, align with, and navigate through a submerged gate.
2. **Slalom Course:** After passing the gate, the submarine must navigate a three-set slalom course, passing on the same side of a center marker pole for each set.

The mission is considered successful upon completion of the final slalom pass.

2.0 Software Architecture & Design

The software is architected with a strong emphasis on modularity and separation of concerns, moving from an initial single-script prototype to a multi-file package structure.

2.1 Project Structure The final project is organized into a main directory and a dedicated ai package:

```
RoboSubSim/
├── main.py
├── simulator.py
├── config.py
├── data_structures.py
├── world.py
├── utils.py
├── ai/
│   ├── __init__.py
│   ├── submarine.py
│   ├── vision.py
│   └── tasks/
│       ├── __init__.py
│       ├── task_base.py
│       ├── gate_task.py
│       ├── victory_dance_task.py
│       ├── slalom_task.py
│       └── hover_task.py
```

2.2 Module Design and Function

- **main.py:** The primary entry point of the application. Its sole responsibility is to assemble the components: it defines the specific sequence of tasks (the mission plan), injects this plan into the Submarine AI engine, injects the AI into the SubmarineSimulator, and starts the simulation.
- **simulator.py:** The simulation environment. The SubmarineSimulator class manages the Pygame window, the main event loop, and user input. It is responsible for applying physics and rendering all world objects.

- **config.py:** A centralized configuration file for all static parameters, tuning gains, color definitions, and HSV color ranges for the vision system.
- **data_structures.py & world.py:** These files define the "data contracts" for the application. world.py defines physical objects like Gate and SlalomPole. data_structures.py defines information passed between modules, such as SensorSuite and ThrusterCommands.
- **ai/submarine.py:** This is the AI "engine" or context class. The Submarine class is not a task itself but the agent that executes the tasks. It provides a library of generic control functions that the tasks can use. It manages the submarine's state variables and progresses through the mission plan.
- **ai/vision.py:** The perception module. This module leverages the OpenCV library for high-performance image processing. Its functions, such as find_blobs_hsv, are responsible for converting the simulator's Pygame surfaces into a format OpenCV can use, performing color-based object detection in the HSV color space, and returning structured data about found objects.
- **ai/tasks/ Package:** This package contains the modular, interchangeable behaviors for the AI. task_base.py defines the abstract Task interface, while the other files contain the specific logic for each high-level behavior (e.g., gate_task.py, slalom_task.py).

2.3 Simulated Vehicle Specifications

- **Physical Dimensions:** Length: 0.457 meters, Width: 0.460 meters, Mass: 4.0 kg.
- **Buoyancy:** The vehicle is slightly negatively buoyant, with a net downward force of 0.4 Newtons.
- **Propulsion System:** The submarine is equipped with a 6-thruster configuration, allowing for control over surge, sway, heave, pitch, and yaw. Four thrusters are mounted horizontally at 45-degree angles for surge, sway, and yaw control. Two thrusters are mounted vertically for heave and pitch control.

- **Sensor Suite:** The AI receives environmental information from a SensorSuite object, which simulates a forward-facing camera, an IMU, a depth sensor, and compass/attitude sensors.

2.4 Simulation-to-Reality Portability

A primary design goal was to ensure the autonomous agent could be deployed on a physical submarine with minimal changes. This is achieved through the SensorSuite and ThrusterCommands data classes, which act as a Hardware Abstraction Layer (HAL).

Path to Deployment:

1. **Simulator-First Validation:** The AI's state machine, control algorithms (including the task-specific PD controller), and OpenCV-based vision logic have been fully developed and validated in the provided simulator.
2. **Create Hardware Interface:** On the physical submarine's companion computer (e.g., a Raspberry Pi), a `hardware_interface.py` module is written to serve as the bridge to the physical world. It reads data from the real IMU, depth sensor, and camera, and sends commands to the thruster ESCs.
3. **Deploy AI:** The entire ai package is copied to the physical submarine.
4. **Integrate AI:** The main execution loop is modified to use the `hardware_interface.py` module to acquire sensor data and send thruster commands, replacing the call to the simulator.
5. **In-Water Tuning:** The system is calibrated in a real-world environment. This involves re-tuning control gains (like the PID values in `submarine.py` and the task-specific gains in `slalom_task.py`) to match the physical submarine's dynamics. Additionally, the HSV color ranges in `config.py` must be recalibrated for the specific lighting conditions and water clarity of the test environment.

3.0 Mission Tasks and Execution Rules

3.1 GateTask

- **Goal:** To find, align with, and pass through the initial submerged gate.
- **Execution:** The sub searches for the gate, aligns by first centering it with yaw and then squaring up with sway, approaches it, and drives through on a timer once it gets close. It then hovers to stabilize before the next task.

3.2 VictoryDanceTask

- **Goal:** To perform a fixed acrobatic routine while maintaining a stable position.
- **Execution:** The sub proceeds through a 16-step sequence of turns and pitches, using gentle, task-specific gains to ensure stability.

3.3 SlalomTask

- **Goal:** To navigate three sets of slalom poles, passing on the same pre-determined side of the center red pole each time.
- **Execution:** This task loops through the following sequence three times:
 1. **SEARCHING:** The sub performs a panoramic sweep to find the poles. It identifies the closest set by finding the group of poles whose bottom edges appear lowest in the camera's view, a method robust to changes in the submarine's pitch.
 2. **ALIGNING:** A precise, two-step process. In YAW_ALIGN, it turns to match the course heading. In SWAY_ALIGN, it uses a task-specific Proportional-Derivative (PD) controller to precisely position the visual midpoint between the red pole and the correct white pole in the center of its view, preventing overshoot.
 3. **APPROACH:** The sub locks its heading and drives straight forward.

4. **CLEAR:** This state is triggered when the submarine gets close enough that the target pole pair can no longer be seen by the vision system, acting as a virtual tripwire. The sub then continues straight on a 3.0-second timer to ensure it has fully passed the obstacle.
5. **STOP:** The sub comes to a complete halt before beginning the search for the next set of poles.

3.4 HoverTask

- **Goal:** To hold a fixed position and heading at the end of the mission.
- **Execution:** The task enters an indefinite loop, calling the main PID hover command to maintain its position. It never completes.
-